SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14 DESIGN COURSE OUTCOME COVERED

COURSE NAME: COMPILER DESIGN

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Annexure A CASE STUDY Total Marks:100

CASE STUDY

Code of Conduct:

I Perarasan V (192521404) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Perarasan V

Annexure A

CASE STUDY

Q1. Both parse trees and syntax trees are used to represent program structure, but differ significantly in complexity and abstraction level. Compare these representations in terms of memory usage, ease of traversal, suitability for optimization, and their role in different phases of compilation such as semantic analysis and code generation.These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, representation choice affects compiler performance. Finally, selecting the right abstraction improves overall efficiency.

1. Compare Parse Trees and Syntax Trees
Comparison

Aspect	Parse Tree	Syntax Tree
Structure	Complete grammar representation	Simplified representation
Memory Usage	High	Low
Traversal	More complex	Easier and faster
Optimization	Difficult	Highly suitable
Semantic Analysis	Useful for syntax verification	Better for type checking
Code Generation	Indirectly used	Directly used

Analysis

- Parse trees preserve every production rule, making them larger and more memory-intensive.
- Syntax trees eliminate redundant nodes such as parentheses and intermediate grammar symbols, reducing storage requirements.
- Compiler optimizations such as **constant folding, dead-code elimination, and common subexpression elimination** operate efficiently on syntax trees.

- During **semantic analysis**, attributes like type, scope, and symbol-table information are attached more conveniently to AST nodes.
- In **code generation**, syntax trees provide a compact structure that can be easily translated into intermediate code and machine instructions.

Conclusion

Parse trees are mainly useful during the **parsing phase** for checking grammatical correctness, while syntax trees are preferred for **semantic analysis, optimization, and code generation** because they improve compiler efficiency and reduce memory overhead.

Q2. A compiler designer must choose between bottom-up and top-down evaluation strategies for implementing syntax-directed definitions. Justify this choice by analyzing grammar restrictions, attribute dependencies, parsing efficiency, and the ease of integrating semantic actions within each approach.These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, evaluation strategy influences implementation complexity. Finally, choosing the right method ensures better performance.

Top-Down Evaluation

Advantages:

- Simple recursive-descent implementation
- Easy handling of inherited attributes
- Suitable for LL grammars

Limitations:

- Cannot directly handle left-recursive grammars
- Less suitable for complex programming-language grammars

Bottom-Up Evaluation

Advantages:

- Works with a larger class of grammars (LR grammars)
- Naturally supports synthesized attributes
- More efficient for practical compiler construction

Limitations:

- Inherited attributes are more difficult to evaluate
- Semantic-action placement is more complex

Attribute Dependency Consideration

For **S-attributed definitions**, bottom-up evaluation is ideal because synthesized attributes can be computed automatically during reductions.

For **L-attributed definitions**, top-down evaluation is often easier because inherited attributes can be passed from parent nodes to child nodes during recursive parsing.

Conclusion

Top-down evaluation is appropriate for **simple grammars and inherited attributes**, whereas bottom-up evaluation is preferred for **real-world compilers** because it supports more expressive grammars and provides efficient parsing. Modern compilers generally adopt **bottom-up LR parsing with carefully scheduled semantic actions**

Q3. In complex arithmetic and logical expressions involving multiple operators, precedence rules, and associativity constraints, attribute dependencies may create conflicts in evaluation order. Analyze how these dependencies affect semantic rule execution and propose techniques such as dependency graphs or evaluation scheduling to resolve them.These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, resolving conflicts is critical for correctness. Finally, structured evaluation improves reliability.

1. Attribute Dependency Conflicts in Expression Evaluation

Example

Expression:

a + b * c - d

The multiplication operation must be evaluated before addition and subtraction because of operator precedence.

Semantic Rules

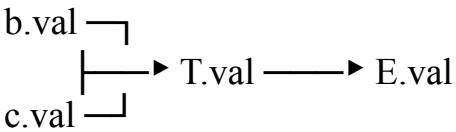
E → E1 + T { E.val = E1.val + T.val }

T → T1 * F { T.val = T1.val * F.val }

Here, `E.val` depends on `T.val`, and `T.val` may depend on further subtree computations. Evaluating attributes in the wrong order can lead to incorrect results or circular dependencies.

Dependency Graph

A dependency graph represents attributes as nodes and dependencies as directed edges:



Evaluation Scheduling

The compiler resolves dependencies using **topological ordering**:

1. Evaluate `b.val` and `c.val`
2. Compute `T.val = b * c`
3. Compute `E.val = a + T.val`
4. Apply subtraction with `d`

Techniques for Conflict Resolution

- Dependency graphs
- Topological sorting
- S-attributed definitions
- L-attributed definitions
- Postorder traversal of syntax trees
- Temporary variables in intermediate code generation

Conclusion

Attribute dependencies directly influence the correctness of semantic-rule execution. Using **dependency graphs and evaluation scheduling** ensures conflict-free attribute computation, preserves operator precedence, and improves compiler reliability and correctness.

Q4. A statically typed programming language enforces type checking during compilation to ensure correctness. Evaluate how such a type system

helps detect errors early, prevents invalid operations, and improves software reliability, while also discussing any limitations in flexibility compared to dynamically typed systems.These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, strong typing enhances safety. Finally, balancing flexibility and strictness is important.

1. Evaluation of a Statically Typed Language

A statically typed programming language performs type checking during compilation, which helps ensure program correctness before execution.

Advantages

1. Early Error Detection
int x = "hello";

The compiler immediately detects the type mismatch during compilation.

2. Prevention of Invalid Operations

Operations such as:

5 + true

can be rejected if the language does not allow implicit conversion between incompatible types.

3. Improved Software Reliability

- Reduces runtime crashes
- Prevents accidental misuse of variables
- Makes large software systems easier to maintain

4. Better Optimization

Because variable types are known at compile time, the compiler can generate more efficient machine code and choose optimized instructions.

Limitations

Reduced Flexibility

Programmers must declare or infer compatible types, which can require additional code and restrict dynamic behavior.

Comparison with Dynamic Typing

Static Typing	Dynamic Typing
Errors detected at compile time	Errors detected at runtime
Faster execution	More flexible
Better optimization	Easier rapid development
Safer for large systems	Convenient for scripting

Conclusion

Static typing improves **safety, correctness, maintainability, and optimization opportunities** by detecting errors early and preventing invalid operations. Although it is less flexible than dynamic typing, it is highly beneficial for developing **reliable and efficient software systems**.

Q5. A modern programming language supports advanced features such as polymorphism, function overloading, and type inference. Analyze the challenges faced by the compiler's type system in resolving types accurately, managing ambiguity, and maintaining efficient compilation performance.These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, advanced features increase complexity. Finally, efficient resolution mechanisms are essential.

1. Challenges in Advanced Type Systems

Modern programming languages support advanced features such as **polymorphism, function overloading, and type inference**. These features improve expressiveness but increase the complexity of the compiler's type system.

1. Polymorphism

Example:

identity(x)

The compiler must determine the actual type of × for each usage while maintaining type safety. This requires **generic type substitution and constraint checking**.

2. Function Overloading

Example:

print(int)
print(string)

The compiler must:

- Select the correct overloaded function
- Handle implicit type conversions
- Avoid ambiguous function calls

For example, `print(5.0)` may match multiple overloads if conversion rules are not clearly defined.

3. Type Inference

Example:

var x = a + b;

The compiler must infer the type of `x` from the operands and operators by collecting and solving **type constraints**.

Compiler Challenges

Feature	Challenge
Polymorphism	Generic type substitution
Function Overloading	Ambiguity resolution
Type Inference	Constraint solving
Combined Features	Increased compilation time

Efficient Resolution Mechanisms

Compilers use several techniques:

- Symbol tables for overloaded-function lookup
- Type constraint graphs
- Unification algorithms for type inference
- Caching and memoization to reduce repeated computations

Conclusion

Advanced type systems make programming languages more **expressive and user-friendly**, but they introduce significant challenges in **type resolution, ambiguity management, dependency handling, and compilation**

performance. Efficient mechanisms such as **constraint solving, unification, and graph-based dependency analysis** are essential for maintaining both correctness and efficient compilation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
		Thorough	Good	Basic	Poor understandi

Understanding of Case	25	understanding; clearly identifies all key issues	understanding with most issues identified	understanding; some issues missed	ng; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand